CHALLENGE SHEET

Threadline

A clothing store REST API. One customer sees more than they should.

DIFFICULTY	TRACK	HOST PORT	LAB ID
MEDIUM	WEB	1000	threadline

EASY · **MEDIUM** · HARD · INSANE

VERIFY THE SOLVE

```
python3 ../scripts/check.py threadline
```

Brief

ThreadLine sells clothing through a REST API. No frontend, no docs. You hold a normal customer account. A flag sits somewhere in the checkout flow.

A customer's account was quietly given a loyalty retention offer after they threatened to close their account. The offer was never meant to be public — get the Premium Leather Jacket for free, the way it was never supposed to happen.

Setup

The API listens on port **`1000`**. All endpoints are under `http://<target-host>:1000/api/v1`. docker compose up –build -d curl -s http://127.0.0.1:1000/api/v1/health

text

Wait for `{"status":"ok"}` before sending further requests.

There is no web frontend — this is an API-only challenge. Use `curl`, `httpie`, Postman, Burp Repeater, or a script; whatever you're comfortable driving raw HTTP requests with.

Setup

1. Register your own account: POST `/api/v1/auth/register` `{ "username": "...", "email": "...", "password": "..." }`
2. Log in — the API uses a session cookie, not a bearer token: POST `/api/v1/auth/login` `{ "email": "...", "password": "..." }`
3. Explore from there. Every account starts with a small balance, but that's not enough to buy anything on its own.

Endpoints (non-exhaustive — some may not matter, some might matter a lot)

Method	Path	Notes
POST	`/api/v1/auth/register`	Create an account
POST	`/api/v1/auth/login`	Get a session cookie
POST	`/api/v1/auth/logout`	Invalidate your session
GET	`/api/v1/users/<id>`	View a user's public profile
GET	`/api/v1/products`	List products
GET	`/api/v1/products/<id>`	View a single product
POST	`/api/v1/coupons`	Request a coupon by email
POST	`/api/v1/cart/items`	Add a product to your cart
POST	`/api/v1/cart/apply-coupon`	Apply a coupon to your cart
POST	`/api/v1/orders`	Checkout your cart
GET	`/api/v1/health`	Liveness check

Goal

Get a cart's `final_price` down to **0**, then check out. If you got there the right way, checkout will hand you a flag. It matches `duck{...}`.

Reset

docker compose restart

text

State resets to the seeded starting point on every restart.

See how to play for the usual solve loop.

Rules

- Don't attack anything outside this container (it has no internet access anyway).
- Don't brute-force passwords or run heavy fuzzing that could take the service down for other players — this is a logic challenge, not a denial-of-service exercise.

Hints

Hint 1

Not every endpoint that requires "being logged in" also checks that you're looking at your own data.

Hint 2

If an endpoint tells you "no offer found for this email," ask yourself exactly how it's comparing the email you sent against the one it has on file.

Hint 3

One coupon might not be enough to hit zero. Is there anything stopping you from having more than one?

Good luck — and remember: the interesting bug is rarely the loudest one.